

# useRef — A Practical Guide

---

A deep-dive covering the two components in this project that use refs ([AutoFocusInput.tsx](#) and [RenderCounter.tsx](#)), the two very different jobs `useRef` does, and the bugs that appear when you confuse it with `useState`.

---

## Table of contents

---

1. What `useRef` is for
  2. Anatomy of the hook
  3. Two jobs, one hook
  4. Case study 1 — `AutoFocusInput` (DOM ref)
  5. Case study 2 — `RenderCounter` (mutable value)
  6. `useRef` vs `useState`
  7. When is `ref.current` actually assigned?
  8. TypeScript with refs
  9. Bug lab — what breaks when you get it wrong
  10. React 18 StrictMode
  11. Rules of thumb
  12. Quick reference table
- 

## What `useRef` is for

---

`useState` gives a component memory that **triggers a re-render** when it changes. But sometimes you need to remember something *without* asking React to redraw — a DOM node, a timer ID, a mutable counter, a previous value.

`useRef` gives a component a **mutable box** that:

- Survives across re-renders.
- Is scoped to *this specific instance* of the component.
- **Does not trigger a re-render** when you change it.
- Is not compared by React — it's just a plain JavaScript object with a `.current` property.

Think of `useRef` as: *"Give me a slot I can write to and read from freely, without React watching."*

---

## Anatomy of the hook

---

```
const ref = useRef<T>(initialValue);
//    ^^^                ^^^^^^^^^^^^^
//    |                    used only on the first render
//    { current: T } - the box React hands you
```

- Returns an object of shape `{ current: T }`.
- The **same object reference** is returned on every render — only `.current` changes.
- `initialValue` is used **only on the first render**. Ignored afterwards.
- Writing `ref.current = x` does **not** re-render the component.
- Reading `ref.current` during render is legal but usually a mistake for DOM refs (see §7).

## Two jobs, one hook

`useRef` is used for two conceptually distinct purposes, and it helps to keep them separate in your head.

### Job A — Hold a reference to a DOM node

You pass the ref to a JSX element's `ref` attribute. After React commits, `ref.current` points to the underlying DOM node.

```
const inputRef = useRef<HTMLInputElement>(null);
// ...


```

Use this when you need to **imperatively** control a DOM node: focus it, measure it, scroll it, play/pause a `<video>`, or hand it to a non-React library.

### Job B — Hold any mutable value across renders

You never touch the `ref` attribute. You just use the box to remember something between renders.

```
const timerId = useRef<number | null>(null);
// timerId.current = setInterval(...);
```

Use this for timer IDs, previous prop values, "did I already run once?" flags, event throttle timestamps, etc.

Both jobs share one rule: **changing `ref.current` never triggers a re-render**.

## Case study 1 — `AutoFocusInput` (DOM ref)

```
import { useEffect, useRef } from "react";

export default function AutoFocusInput() {
  const inputRef = useRef<HTMLInputElement>(null);

  useEffect(() => {
    inputRef.current?.focus();
  }, []);

  return (
    <>
      <label htmlFor="auto">Auto-Focused</label>
      <input id="auto" type="text" ref={inputRef} placeholder="Focus on Mount" />
    </>
  );
}
```

## Why `useRef` and not `useState` ?

We don't need React to re-render when we get a handle to the input — we just need the handle so we can call `.focus()` on it. `useState` would work but every change to `.current` would waste a render.

## Why `useEffect(..., [])` instead of calling `.focus()` in the body?

React's render is a **description** of the UI, not the UI itself. When the component function runs, the `<input>` element doesn't exist in the DOM yet — it's still a virtual node. `inputRef.current` is still `null` during render.

Timeline on mount:

- |                    |                                                                                                           |
|--------------------|-----------------------------------------------------------------------------------------------------------|
| 1. Render phase    | → function runs, JSX returned. <code>inputRef.current === null</code> .                                   |
| 2. Commit phase    | → React creates the real <code>&lt;input&gt;</code> , attaches it to refs.                                |
| 3. Layout effects  | → <code>useLayoutEffect</code> callbacks run.                                                             |
| 4. Paint           | → browser paints the screen.                                                                              |
| 5. Passive effects | → <code>useEffect</code> callbacks run. <code>inputRef.current</code> is the <code>&lt;input&gt;</code> . |

`.focus()` needs the DOM node to exist, so it belongs in step 5.

## Why `?.` (optional chaining)?

TypeScript sees `inputRef` as `RefObject<HTMLInputElement>` whose `.current` is `HTMLInputElement | null`. `.focus()` on `null` would crash. The `?.` says "only call if not null." In practice, by the time this effect runs, `.current` is never null — but the type system doesn't know that.

## Why the empty deps array?

We want the focus to happen once, on mount. Not on every re-render. `[]` means "run this effect exactly once after the first commit."

---

## Case study 2 — `RenderCounter` (mutable value)

---

```
import { useRef, useState, useEffect } from "react";

export default function RenderCounter() {
  const [count, setCount] = useState(0);
  const renderCount = useRef(0);

  useEffect(() => {
    renderCount.current += 1;
  });

  return (
    <div>
      <p>Button clicks (state): {count}</p>
      <p>Renders (ref): {renderCount.current}</p>
      <button onClick={() => setCount((c) => c + 1)}>Increment</button>
    </div>
  );
}
```

## Why `useRef` and not `useState` for the render count?

If we did `setRenderCount(x + 1)` during render, it would trigger *another* render, which would call `setRenderCount` again — infinite loop. Refs let us tally renders **without feeding React more work to do**.

## Why the effect has no deps array?

We want to increment on **every** committed render, not just on mount and not just when a specific value changes. Omitting the array = "run after every render." That's rare and usually a red flag, but it's exactly what we want here.

## Why the number on screen is always "one behind"

The order per render is: **render** → **commit** → **paint** → **effect**. So:

1. Render reads `renderCount.current` (say `0`) and produces JSX showing `0`.
2. React commits and paints — screen shows `0`.

3. Effect runs, sets `renderCount.current = 1`.
4. But no re-render was triggered — screen stays on `0` until *something else* (like a click) causes a re-render.
5. Next click → render reads `1`, displays `1`, effect bumps to `2`, screen stays on `1`.

So the displayed value = "how many renders **committed before this one.**"

## The "impure" alternative

If you mutate the ref **during render** instead of in an effect, the displayed number matches the current render:

```
renderCount.current += 1; // during render – side effect in render body
```

This is technically impure (render should have no side effects), and StrictMode will punish you by doubling the count in dev. See §10.

## useRef VS useState

| Aspect                                    | useState                                   | useRef                          |
|-------------------------------------------|--------------------------------------------|---------------------------------|
| Return shape                              | <code>[value, setter]</code>               | <code>{ current: value }</code> |
| Triggers re-render on change?             | <b>Yes</b>                                 | <b>No</b>                       |
| Update semantics                          | Asynchronous, batched, snapshot per render | Synchronous, immediate mutation |
| Right for values shown in the UI          | ✓                                          | ✗                               |
| Right for values you read during render   | ✓                                          | ⚠ Only for DOM refs after mount |
| Right for timer IDs, subscription handles | ✗ (would over-render)                      | ✓                               |
| Right for "previous value" tracking       | ✗                                          | ✓                               |
| Right for DOM node handles                | ✗ (can't hold nodes)                       | ✓                               |

## Rule of thumb

If the UI needs to visually change when this value changes, use `useState`. Otherwise, `useRef`.

## When is `ref.current` actually assigned?

For **DOM refs**, this is the most important timing question. React assigns `ref.current` during the **commit phase**, *after* your component function has returned its JSX and *before* effects run.

|                                        |                                                                                   |
|----------------------------------------|-----------------------------------------------------------------------------------|
| Component function runs                | → <code>ref.current</code> is still whatever it was before (null on first render) |
| React commits DOM                      | → <code>ref.current</code> = the DOM node                                         |
| <code>useLayoutEffect</code> callbacks | → <code>ref.current</code> is the node (blocks paint)                             |
| Browser paints                         | → user sees the update                                                            |
| <code>useEffect</code> callbacks       | → <code>ref.current</code> is the node (runs after paint)                         |



Consequences:

- **You cannot read a DOM ref during render.** It's still `null` (on first render) or stale (on subsequent).
- **`useLayoutEffect` is the earliest safe place** to read a DOM ref and mutate the DOM before the user sees anything (e.g., measuring layout to position a tooltip).
- **`useEffect` is the normal place** to interact with a DOM ref for focus, event wiring, non-visual work.

For **non-DOM refs** (Job B), `.current` holds whatever you last wrote to it. No lifecycle mystery — it's just an object property.

## TypeScript with refs

### DOM refs — start with `null`

```
const inputRef = useRef<HTMLInputElement>(null);
// inputRef.current: HTMLInputElement | null
```

The type argument is the element type. Common ones: `HTMLInputElement`, `HTMLButtonElement`, `HTMLDivElement`, `HTMLTextAreaElement`, `HTMLVideoElement`. You must access via `?.` or a null check.

### Mutable value refs — start with the actual value

```
const count = useRef<number>(0);
// count.current: number (no null in the union)
```

Because the initial value is `0`, not `null`, TypeScript knows `.current` is always a `number`.

## Timer IDs

```
const timerId = useRef<number | null>(null);
// later:
timerId.current = window.setInterval(tick, 1000);
// cleanup:
if (timerId.current !== null) clearInterval(timerId.current);
```

Note: in browsers, `setInterval` returns `number` ; in Node it returns `NodeJS.Timeout` . Use `window.setInterval` in browser code to get the browser-typed overload.

## Bug lab — what breaks when you get it wrong

### Bug 1 — Reading a DOM ref during render

```
export default function Broken() {
  const inputRef = useRef<HTMLInputElement>(null);
  inputRef.current?.focus(); // ❌ runs during render, .current is null on first render
  return <input ref={inputRef} />;
}
```

**What happens:** On first render, `.current` is `null` , the optional chain silently skips the focus, and the input never focuses. On re-renders, the focus *does* fire — meaning the input steals focus every time the parent re-renders. Move it to `useEffect(..., [])` .

### Bug 2 — Using a ref for something the UI needs to display

```
const nameRef = useRef("");
return (
  <>
    <input onChange={e => {nameRef.current = e.target.value}} />
    <p>Hello, {nameRef.current}</p> /* ❌ never updates */
  </>
);
```

**What happens:** Typing updates `.current` but no re-render happens, so the `<p>` never re-runs and the screen never shows the new name. Rule: if the render reads it and users need to see changes, it must be state.

### Bug 3 — Using state for something the UI doesn't display

```
const [timerId, setTimerId] = useState<number | null>(null);
useEffect(() => {
  const id = window.setInterval(tick, 1000);
  setTimerId(id); // ✗ triggers an extra render for no visual benefit
  return () => { if (timerId !== null) clearInterval(timerId); }; // stale closure!
}, []);
```

**What happens:** You over-render, and the cleanup captures the *initial* `timerId` ( `null` ) instead of the one you just set. Refs would fix both problems:

```
const timerId = useRef<number | null>(null);
useEffect(() => {
  timerId.current = window.setInterval(tick, 1000);
  return () => { if (timerId.current !== null) clearInterval(timerId.current); };
}, []);
```

## Bug 4 — Mutating a ref during render (impurity)

```
export default function Counter() {
  const renders = useRef(0);
  renders.current += 1; // ✗ side effect during render
  return <p>{renders.current}</p>;
}
```

**What happens:** In development with StrictMode, React invokes the render twice, so `renders.current` grows twice as fast as it should. In concurrent rendering, React may throw away a render entirely — your counter would over-count. Move the mutation to an effect if you can, or accept the impurity for debug-only counters.

## Bug 5 — Expecting the ref to be a dependency

```
useEffect(() => {
  console.log(inputRef.current);
}, [inputRef.current]); // ✗ meaningless
```

**What happens:** `inputRef.current` is read once when the render runs — before commit — so it's `null` (or stale). React never "sees" `.current` change because refs aren't reactive. Depending on `.current` doesn't do what you want. If you truly need to react to a DOM node appearing, use a **callback ref**:



```
const setRef = useCallback((node: HTMLInputElement | null) => {
  if (node) node.focus();
}, []);
return <input ref={setRef} />;
```

## React 18 StrictMode

In development, `<React.StrictMode>` intentionally **double-invokes** your component function on every render (throwing away the first result) to help you catch impure render logic.

For refs, this matters when you **mutate a ref during render**:

```
renderCount.current += 1; // in render body
```

- Without StrictMode: counts 1, 2, 3, 4, ...
- With StrictMode (dev only): counts 2, 4, 6, 8, ...

The doubling is the tell-tale sign that render isn't pure. StrictMode isn't broken — your render is. Move the mutation to `useEffect` if the count needs to be accurate, or accept the doubling as the intended lesson.

For **DOM refs**, StrictMode also runs mount effects twice ( `setup` → `cleanup` → `setup` ). The DOM node itself doesn't change between the two invocations, so `.current` stays consistent — but any imperative work you do (like `.focus()` ) will fire twice. That's usually fine; `.focus()` is idempotent.

**This doubling does not happen in production.**

## Rules of thumb

1. **If the UI needs to update, use `useState` . If not, consider `useRef` .**
2. **Don't read a DOM ref during render.** Read it in `useEffect` (or `useLayoutEffect` if you need it before paint).
3. **Don't mutate refs during render** except for debug-only counters. Render should be pure.
4. **Refs are not reactive.** Listing `ref.current` in a dependency array does nothing useful.
5. **The initial value passed to `useRef` is used only once.** Don't put expensive computations there — pass a value, not a call to a heavy function.
6. **For DOM refs, type as `useRef<HTMLXxxElement>(null)` .** Access via `?.` or a null check.
7. **For mutable value refs, type with the actual value type.** No `null` in the union unless you truly need it.
8. **Prefer callback refs** when you need to react to a node being attached or detached.

## Quick reference table

| Use case                                            | Hook                                        | Initial value      | Read during render?                   |
|-----------------------------------------------------|---------------------------------------------|--------------------|---------------------------------------|
| Focus an input on mount                             | <code>useRef&lt;HTMLInputElement&gt;</code> | <code>null</code>  | ✗ (use effect)                        |
| Measure a DOM element's size                        | <code>useRef&lt;HTMLDivElement&gt;</code>   | <code>null</code>  | ✗ (use <code>useLayoutEffect</code> ) |
| Store a <code>setInterval</code> ID                 | <code>useRef&lt;number   null&gt;</code>    | <code>null</code>  | ✓ (it's just a value)                 |
| Track previous prop value                           | <code>useRef&lt;T&gt;</code>                | current prop       | ✓ (in an effect that runs after)      |
| Count renders (debug)                               | <code>useRef&lt;number&gt;</code>           | <code>0</code>     | ✓ (accepting impurity)                |
| Skip effect on first mount                          | <code>useRef&lt;boolean&gt;</code>          | <code>false</code> | ✓                                     |
| Cache a heavy computed value that never triggers UI | <code>useRef&lt;T   null&gt;</code>         | <code>null</code>  | ✓                                     |
| Any value the UI must display                       | ✗ use <code>useState</code> instead         | —                  | —                                     |